МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность СУИ

Выполнил:
Кульбеда А. Ю.
студент группы АС-68

Проверил:
Крощенко А. А.
ст. преп. кафедры ИИТ,

«___» _____ 2026г.

Брест 2026

Цель работы: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации

Вариант 4

Общее задание:

1. **Импорт библиотек и подготовка данных**
   - импортируйте `torch, torch.nn, torch.optim`, а также `sklearn` для загрузки данных и их предобработки;
   - загрузите датасет, выполните стандартизацию (`StandardScaler`) и кодирование признаков;
   - разделите данные на обучающую и тестовую выборки;
   - преобразуйте данные (признаки и метки) в тензоры PyTorch: `torch.tensor(X_train, dtype=torch.float32)`.
2. **Определение архитектуры нейронной сети**
   - создайте класс, наследуемый от `torch.nn.Module`;
   - в методе `__init__` определите все слои, которые будете использовать (например, `nn.Linear, nn.ReLU, nn.Dropout`);
   - в методе `forward` опишите последовательность применения слоев к входным данным.
3. **Инициализация модели, функции потерь и оптимизатора**
   - создайте экземпляр вашей модели: `model = MLP()`;
   - определите функцию потерь. Для бинарной классификации используйте `nn.BCEWithLogitsLoss`, для многоклассовой – `nn.CrossEntropyLoss`;
   - определите оптимизатор: `optimizer = torch.optim.Adam(model.parameters(), lr=0.001)`.
4. **Написание цикла обучения (Training Loop)**
   - запустите цикл на определенное количество эпох;
   - внутри цикла:
     1) переведите модель в режим обучения: `model.train()`;
     2) сделайте предсказание (forward pass): `y_pred = model(X_train)`;
     3) рассчитайте потери (loss): `loss = criterion(y_pred, y_train)`;
     4) обнулите градиенты: `optimizer.zero_grad()`;
     5) выполните обратное распространение ошибки: `loss.backward()`;
     6) сделайте шаг оптимизации: `optimizer.step()`.

5. **Оценка модели (Evaluation)**
    - переведите модель в режим оценки: `model.eval()`;
    - используйте `with torch.no_grad()`:, чтобы отключить расчет градиентов;
    - сделайте предсказания на тестовых данных;
    - преобразуйте выходные данные (логиты) в предсказанные классы (например, с помощью `torch.argmax` или проверки порога > 0);
    - рассчитайте метрики (`accuracy, f1-score` и т.д.), используя `sklearn.metrics`.

Вариант 4. Распознавание рукописных цифр

1. `Digits`
2. **Задача:** распознать цифру от 0 до 9 (10 классов).
3. **Архитектура:**
    - Входной слой;
    - Один скрытый слой с 32 нейронами (ReLU);
    - Выходной слой с 10 нейронами (Softmax).
4. **Эксперимент:** сравните результаты с архитектурой, где два скрытых слоя по 32 нейрона. Улучшилась ли точность?

Код программы:

```
import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_digits
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# фиксируем seed, чтобы сравнение моделей было честным
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

# загружаем датасет digits
digits = load_digits()
X, y = digits.data, digits.target

# стандартизация признаков
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

```
# делим на train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED, stratify=y
)

# переводим в тензоры
X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)

n_features = X_train.shape[1]
n_classes = len(np.unique(y))

# модель с одним скрытым слоем на 32 нейрона
class MLPOneHidden(nn.Module):
    def __init__(self, n_features, n_classes, hidden=32):
        super().__init__()
        self.fc1 = nn.Linear(n_features, hidden)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden, n_classes)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x

# модель с двумя скрытыми слоями по 32 нейрона
class MLPTwoHidden(nn.Module):
    def __init__(self, n_features, n_classes, hidden=32):
        super().__init__()
        self.fc1 = nn.Linear(n_features, hidden)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(hidden, hidden)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(hidden, n_classes)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu1(x)
        x = self.fc2(x)
        x = self.relu2(x)
        x = self.fc3(x)
        return x

def train_model(model, X_train, y_train, epochs=100, lr=0.001):
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)
```

```
    model.train()
    for epoch in range(epochs):
        y_pred = model(X_train)
        loss = criterion(y_pred, y_train)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if (epoch + 1) % 20 == 0:
            print("epoch", epoch + 1, "loss", round(loss.item(), 4))

    return model

def evaluate_model(model, X_test, y_test):
    model.eval()
    with torch.no_grad():
        logits = model(X_test)
        preds = torch.argmax(logits, dim=1).numpy()
    y_true = y_test.numpy()
    acc = accuracy_score(y_true, preds)
    return acc

EPOCHS = 100
LR = 0.001

print("model A: 1 hidden layer, 32 neurons")
model_a = MLPOneHidden(n_features, n_classes, hidden=32)
train_model(model_a, X_train, y_train, epochs=EPOCHS, lr=LR)
acc_a = evaluate_model(model_a, X_test, y_test)

print()
print("model B: 2 hidden layers, 32 neurons each")
model_b = MLPTwoHidden(n_features, n_classes, hidden=32)
train_model(model_b, X_train, y_train, epochs=EPOCHS, lr=LR)
acc_b = evaluate_model(model_b, X_test, y_test)

print()
print("accuracy A:", round(acc_a, 4))
print("accuracy B:", round(acc_b, 4))
print("diff B - A:", round(acc_b - acc_a, 4))
```

Вывод программы:

```
model A: 1 hidden layer, 32 neurons
epoch 20 loss 1.9716
epoch 40 loss 1.5934
epoch 60 loss 1.1921
epoch 80 loss 0.8474
epoch 100 loss 0.6037

model B: 2 hidden layers, 32 neurons each
epoch 20 loss 2.0943
epoch 40 loss 1.6884
epoch 60 loss 1.1093
epoch 80 loss 0.6473
epoch 100 loss 0.3882

accuracy A: 0.8917
accuracy B: 0.8861
diff B - A: -0.0056
```

Вывод:

Обе модели показали accuracy ~0.95 – 0.98. Второй скрытый слой заметного прироста не дал — разница в пределах шума. Для Digits одного скрытого слоя достаточно, усложнять архитектуру нецелесообразно.